

CONTENTS

Semana 3 — Álgebra de Boole, Método de Karnaugh, Circuitos Combinatorios	
Arquitectura de Computadoras — FING UdelaR · Guía de la semana	
Checklist — qué hay que saber	
Teoría condensada (notación del curso)	
Ejercicios del Práctico 3 — tabla de dificultad	
Dónde aparece este tema en parciales y exámenes anteriores	

Semana 3 — Álgebra de Boole, Método de Karnaugh, Circuitos Combinatorios

ARQUITECTURA DE COMPUTADORAS — FING UDELAR · GUÍA DE LA SEMANA

Checklist — qué hay que saber

- ☐ Axiomas del álgebra de Boole (neutro, complemento, conmutativa, distributiva, asociativa, ax.5/ax.6 del curso) y los teoremas derivados (idempotencia, absorción, De Morgan).
- ☐ Representación Σ (suma de productos canónicos, minterms) y Π (producto de sumas canónicas, maxterms) — y la relación entre ambas.
- ☐ Método de Karnaugh: agrupar 1's (o 0's) en potencias de 2 adyacentes, leer términos por las variables que NO cambian dentro del grupo.
- ☐ Saber que el mínimo en dos niveles **no es único** — puede haber más de una expresión mínima válida para la misma tabla.
- ☐ Circuitos combinatorios estándar: decodificador, multiplexor, demultiplexor, comparador, compuertas NAND/NOR como conjunto universal.
- ☐ Técnica de “MUX como look-up table”: un multiplexor de $2^n:1$ puede implementar cualquier función de n variables usando las variables como líneas de selección y los valores de la tabla de verdad como entradas de datos.

Teoría condensada (notación del curso)

Compuerta básica: conectiva binaria usada para implementar circuitos (AND, OR, NAND, NOR, XOR, NOT).

Σ (suma de productos canónicos): para cada fila donde $f=1$, se arma un producto (minterm) con todas las variables — sin complementar si valen 1, complementadas si valen 0 — y se suman todos esos productos.

Π (producto de sumas canónicas): análogo pero sobre las filas donde $f=0$, con las variables sin complementar si valen 0 y complementadas si valen 1, y se multiplican las sumas (maxterms).

Karnaugh — regla clave: agrupar celdas con 1 en bloques de tamaño 1, 2, 4, 8... (potencias de 2) adyacentes (incluyendo wrap-around de bordes). Cada grupo da un término con las variables que se mantienen constantes en todas las celdas del grupo. **Cuidado:** minimiza la cantidad de términos/literales de la expresión — **no** garantiza la cantidad mínima de compuertas físicas del circuito final (eso depende de reutilización de subtérminos y la tecnología de compuertas disponible).

MUX como generador de funciones: dado un multiplexor de $2^n:1$, conectando las n variables de la función a las líneas de selección y cada entrada de datos D_i al valor de la fila i de la tabla de verdad (0, 1, la variable restante, o su complemento), se puede implementar *cualquier* función booleana sin usar compuertas lógicas — solo el MUX.

Ejercicios del Práctico 3 — tabla de dificultad

Ejercicio	Tema	Dificultad (★ del práctico)	Notas
Teóricas (a-e)	Definiciones + verdadero/falso sobre Karnaugh	—	Conceptual, corto
1, 2	Demostraciones algebraicas con axiomas/teoremas	★★	Hay que citar el axioma/teorema exacto en cada paso
3	Suma de productos canónicos a partir de Σ/Π	★	Directo de la tabla de verdad, sin simplificar
4	Simplificación con Karnaugh (3 funciones)	★	Karnaugh de 3-4 variables
5	Modelado + simplificación (problema de póliza de seguro)	★★	Traducir enunciado a booleano antes de simplificar
6	No unicidad del mínimo en 2 niveles	★	Ejemplo con Karnaugh de 4 variables, dos agrupamientos válidos distintos
7	Equivalencia de dos circuitos (De Morgan)	★★	Reducir ambas expresiones a una forma común
8	Decodificador $4 \rightarrow 16$	★	AND's + NOT's de las 4 entradas
9	Módulo 16 de un entero de 32 bits	★	Truco: es solo cableado directo de los 4 bits menos significativos
10	AND y OR con NAND únicamente	★★	Compuertas universales
11	Decodificador 7 segmentos	★★	(a) con decodificador+OR; (b) expresión mínima por segmento — completado acá, el material original solo resolvía el segmento “a”
12	$f(x)=-x$ con multiplexores únicamente	★★★	Look-up table de 8 filas, 3 bits complemento a 2

Ejercicio	Tema	Dificultad (★ del práctico)	Notas
13	Comparador de 4 bits sin multiplexores	★★★	Construcción en cascada bit a bit
14	Karnaugh de 5 variables (5 funciones)	★★	Verificado computacionalmente (Quine-McCluskey + verificación cruzada), ver nota en el resuelto

(Resolución completa y verificada paso a paso: ver [Practico3_Resolucion.pdf](#) en [Guías de Estudio Arquitectura de Computadoras](#).)

Nota sobre el método de verificación: los ejercicios 11(b), 12, 13 y 14 (los que no tenían solución en el material del curso, o la tenían incompleta) se resolvieron y además se **verificaron computacionalmente** con dos algoritmos de minimización booleana independientes (un Quine-McCluskey propio y `sympy.logic.SOPform`), chequeando que la expresión resultante cubra exactamente — ni de más ni de menos — los minterms pedidos en los 32 (o 16) puntos de la tabla de verdad completa. Donde los dos métodos dieron agrupamientos distintos pero de igual costo (mismo número de términos y literales), se lo marca explícitamente — es el fenómeno del Ejercicio 6 (el mínimo en dos niveles no es único).

Dónde aparece este tema en parciales y exámenes anteriores

Según el relevamiento ya hecho sobre los parciales/exámenes de ArqComp (ver [plan_de_estudio.md](#), sección “Índice de Parciales anteriores” e “Índice de Exámenes anteriores”): Álgebra de Boole y circuitos combinatorios son **consistentemente una pregunta corta** en casi todos los parciales (simplificación por Karnaugh, construcción de compuertas universales con NAND, identificar la función de un multiplexor fijo, construir XOR/XNOR), pero **rara vez el problema grande** del examen — a diferencia de M6 (8086) o M10 (caché), que sí dominan los ejercicios grandes. Conviene dominar bien el mecanismo de Karnaugh y las compuertas universales, pero no es donde más tiempo de práctica activa conviene invertir en términos relativos.